

Kudaibergen Aibyn

ID: 25B031353

Practice Tuesday 11:00-12:00

Normalization

1. Consider the following unnormalized table used for printing booking receipts: `Booking_receipt(booking_id, passenger_full_name, passenger_passport_number, flight_number, departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name, seat_numbers, ticket_price)`. The `seat_numbers` column stores a comma-separated list (e.g. "12A, 12B") when a booking covers several passengers on the same reservation. Identify which normal form (1NF, 2NF, 3NF) this table violates and explain why, referring to the specific columns responsible.
2. Normalize `Booking_receipt` step by step: first bring it to 1NF, then to 2NF, then to 3NF. For each step draw/describe the resulting table(s), underline the primary key of each table, and mark the foreign keys.
3. List the functional dependencies you used to justify each normalization step in task 2 (e.g. `booking_id → flight_number`, `flight_number → airline_name`).
4. For the airport database you created in the DDL section, check whether the `Flights` table is in 3NF. State the current primary key and functional dependencies and explain whether any transitive dependency exists between non-key attributes (for example, via `departing_airport_id` or `airline_id`).
5. Using `CREATE TABLE` statements, implement the 3NF tables you designed in task 2, then write `INSERT` statements that populate them with at least 5 rows of sample data split out from a denormalized dataset of your choice.

DDL

1. Create following tables with corresponding attributes:
 - **Airline_info:** `airline_id (int), airline_code (varchar (30)), airline_name (varchar(50)), airline_country (varchar(50)), created_at(timestamp), updated_at(timestamp), info (varchar(50)) ;`
 - **Airport:** `airport_id(int), airport_name (varchar(50)), country (varchar(50)), state (varchar(50)), city (varchar(50)), created_at(timestamp), updated_at(timestamp) ;`
 - **Baggage_check:** `baggage_check_id(int), check_result (varchar(50)), created_at(timestamp), updated_at(timestamp), booking_id(int), passenger_id(int);`
 - **Baggage:** `baggage_id(int), weight_in_kg (decimal(4,2)), created_at(timestamp), updated_at(timestamp) , booking_id(int);`
 - **Boarding_pass:** `boarding_pass_id(int), booking_id(int), seat (varchar(50)), boarding_time(timestamp), created_at(timestamp), updated_at(timestamp);`

- **Booking_flight:** booking_flight_id(int), booking_id(int), flight_id(int), created_at(timestamp), updated_at(timestamp);
 - **Booking:** booking_id(int), flight_id(int), passenger_id(int), booking_platform(varchar(50)), created_at(timestamp), updated_at(timestamp), status(varchar(50)), price(decimal(7,2));
 - **Flights:** flight_id(int), sch_departure_time(timestamp), sch_arrival_time(timestamp), departing_airport_id(int), arriving_airport_id(int), departing_gate(varchar(50)), arriving_gate(varchar(50)), airline_id(int), act_departure_time(timestamp), act_arrival_time(timestamp), created_at(timestamp), updated_at(timestamp) ;
 - **Passengers:** passenger_id(int), first_name(varchar(50)), last_name(varchar(50)), date_of_birth(date), gender(varchar(50)), country_of_citizenship(varchar(50)), country_of_residence(varchar(50)), passport_number(varchar(20)), created_at(timestamp), updated_at(timestamp) ;
 - **Security_check:** security_check_id(int), check_result(varchar(20)), created_at(timestamp), updated_at(timestamp), passenger_id(int);
2. Define Primary Keys for each tables;
 4. Define for all attributes not null constraint;
 5. Rename airline_info table to airline;
 6. Rename column price to ticket_price in booking table;
 7. Change data type of departing_gate from varchar(50) to text;
 8. Drop the column info(varchar(50)) from the airline table.
 9. Make a relationship between following tables:
 - Passengers with Secuitiry_check, Booking, Baggage_check by passenger_id;
 - Booking with Baggage_check, Baggage, Boarding_pass, Booking_flight by booking_id;
 - Flights with Booking_flight by flight_id;
 - Airport with Flights by departing_airport_id;
 - Airport with Flights by arriving_airport_id;
 - Airline with Flights by airline_id;

First, to start normalization I created a table with all essential attributes and entities:

The screenshot shows the pgAdmin 4 interface. On the left, the 'Object Explorer' pane shows the database structure. The 'public' schema is selected, and the 'Tables' folder is expanded, showing the 'booking_receipt_inf' table. The main pane displays the SQL query for creating the table:

```
1 CREATE TABLE Booking_receipt_INF (
2   booking_id INT NOT NULL,
3   passenger_passport_number VARCHAR(20) NOT NULL,
4   passenger_full_name VARCHAR(100) NOT NULL,
5   flight_number VARCHAR(20) NOT NULL,
6   departure_airport_name VARCHAR(50) NOT NULL,
7   departure_city VARCHAR(50) NOT NULL,
8   arrival_airport_name VARCHAR(50) NOT NULL,
9   arrival_city VARCHAR(50) NOT NULL,
10  airline_name VARCHAR(50) NOT NULL,
11  seat_number VARCHAR(10) NOT NULL,
12  ticket_price DECIMAL(7,2) NOT NULL,
13
14  PRIMARY KEY (booking_id, passenger_passport_number)
15 );
```

The screenshot shows the 'Data Output' pane with the table structure. The columns and their data types are:

booking_id	passenger_passport_number	passenger_full_name	flight_number	departure_airport_name	departure_city	arrival_airport_name	arrival_city	airline_name	seat_number
[PK] integer	[PK] character varying (20)	character varying (100)	character varying (20)	character varying (50)	character varying (50)	character varying (50)	character varying (50)	character varying (50)	character varying (10)

After that, I added some values inside the table:

Query

Query History

1

INSERT INTO Booking_receipt_INF VALUES

2

(101, 'P12345', 'John Smith', 'FL100',

3

'Almaty Airport', 'Almaty',

4

'Heathrow Airport', 'London',

5

'Air Kazakhstan', '12A', 500.00);

6

7

INSERT INTO Booking_receipt_INF VALUES

8

(101, 'P67890', 'Mary Smith', 'FL100',

9

'Almaty Airport', 'Almaty',

10

'Heathrow Airport', 'London',

11

'Air Kazakhstan', '12B', 500.00);

Scratch Pad

Data Output

Messages

Notifications

Showing rows: 1 to 2

Page No: 1 of 1

booking_id	passenger_passport_number	passenger_full_name	flight_number	departure_airport_name	departure_city	arrivalAirport_name	arrivalCity	airline_name	seat_number
1	101	P12345	Walter White	FL100	ALA	Almaty	Frankfurt Airport	London	Air Astana
2	101	P67890	Saul Goodman	FL100	ALA	Almaty	Frankfurt Airport	London	Air Astana

Total rows: 2

Query complete 00:00:00.110

CRLF

Ln 11, Col 35

This is part of 1NF normalization, since I stored “12A and 12B” in a different column named “seat_number”

After the 1NF part completed, I removed partial dependencies, by separating columns into own tables, to fulfil 2NF conditions:

```
Query  Query History
1  CREATE TABLE Passenger_2NF (
2      passenger_passport_number VARCHAR(20) NOT NULL,
3      passenger_full_name VARCHAR(100) NOT NULL,
4
5      PRIMARY KEY (passenger_passport_number)
6  );
```

```
Query  Query History
1  CREATE TABLE Booking_2NF (
2      booking_id INT NOT NULL,
3      flight_number VARCHAR(20) NOT NULL,
4      ticket_price DECIMAL(7,2) NOT NULL,
5
6      PRIMARY KEY (booking_id)
7  );
```

```
Query  Query History
1  CREATE TABLE Booking_Passenger_2NF (
2      booking_id INT NOT NULL,
3      passenger_passport_number VARCHAR(20) NOT NULL,
4      seat_number VARCHAR(10) NOT NULL,
5
6      PRIMARY KEY (booking_id, passenger_passport_number),
7
8      FOREIGN KEY (booking_id)
9          REFERENCES Booking_2NF(booking_id),
10
11     FOREIGN KEY (passenger_passport_number)
12         REFERENCES Passenger_2NF(passenger_passport_number)
13 );
```

Here I made 3 new tables – “passenger”; “booking”, “booking_passenger”, assigned all PK’s and FK’s, and established relationships between them.

To start going from 2NF to 3NF, I removed more dependencies via separating flight, airport, airline into separate tables:

Airline:

Query	Query History
1	CREATE TABLE Airline_3NF (
2	airline_name VARCHAR(50) NOT NULL,
3	
4	PRIMARY KEY (airline_name)
5	);

Airport:

Query	Query History
1	CREATE TABLE Airport_3NF (
2	airport_name VARCHAR(50) NOT NULL,
3	city VARCHAR(50) NOT NULL,
4	
5	PRIMARY KEY (airport_name)
6	);

Flights:

Query	Query History
1	CREATE TABLE Flight_3NF (
2	flight_number VARCHAR(20) NOT NULL,
3	departure_airport_name VARCHAR(50) NOT NULL,
4	arrival_airport_name VARCHAR(50) NOT NULL,
5	airline_name VARCHAR(50) NOT NULL,
6	
7	PRIMARY KEY (flight_number),
8	
9	FOREIGN KEY (departure_airport_name)
10	REFERENCES Airport_3NF(airport_name),
11	
12	FOREIGN KEY (arrival_airport_name)
13	REFERENCES Airport_3NF(airport_name),
14	
15	FOREIGN KEY (airline_name)
16	REFERENCES Airline_3NF(airline_name)
17	);

I established relationships with “booking” and “flights” table:

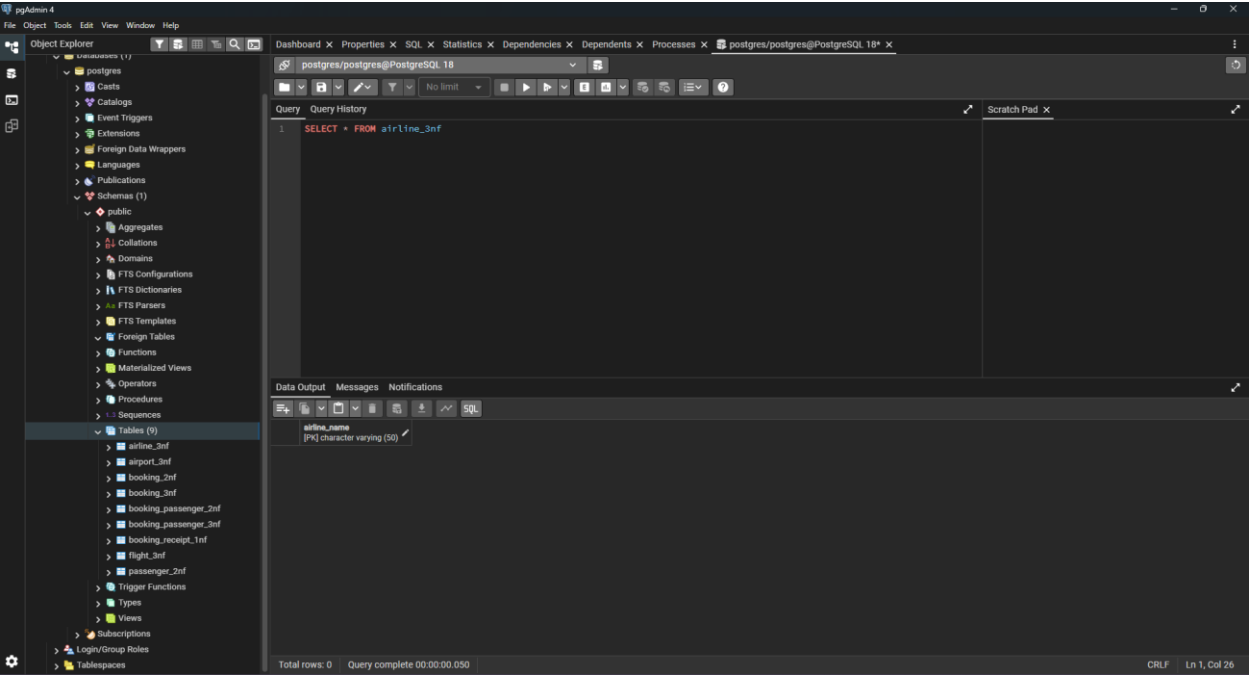
Query	Query History
1	CREATE TABLE Booking_3NF (
2	booking_id INT NOT NULL,
3	flight_number VARCHAR(20) NOT NULL,
4	ticket_price DECIMAL(7,2) NOT NULL,
5	
6	PRIMARY KEY (booking_id),
7	
8	FOREIGN KEY (flight_number)
9	REFERENCES Flight_3NF(flight_number)
10	);

And another relationship for “passengers” and “booking”:

Query	Query History
1	CREATE TABLE Booking_Passenger_3NF (
2	booking_id INT NOT NULL,
3	passenger_passport_number VARCHAR(20) NOT NULL,
4	seat_number VARCHAR(10) NOT NULL,
5	
6	PRIMARY KEY (booking_id, passenger_passport_number),
7	
8	FOREIGN KEY (booking_id)
9	REFERENCES Booking_3NF(booking_id),
10	
11	FOREIGN KEY (passenger_passport_number)
12	REFERENCES Passenger_2NF(passenger_passport_number)
13	);

Now we can see all the normalization steps and corresponding tables:

3NF:



airport_name [PK] character varying (50)	city character varying (50)
--	---------------------------------------

Data Output Messages Notifications		
booking_id [PK] integer	flight_number character varying (20)	ticket_price numeric (7,2)

Data Output Messages Notifications		
booking_id [PK] integer	passenger_passport_number [PK] character varying (20)	seat_number character varying (10)

Data Output Messages Notifications			
flight_number [PK] character varying (20)	departure_airport_name character varying (50)	arrival_airport_name character varying (50)	airline_name character varying (50)

2NF:

Data Output	Messages	Notifications
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL		
	booking_id [PK] integer	flight_number character varying (20)
		ticket_price numeric (7,2)

Here, in the table “booking” according to assignment task, I renamed column price to “ticket_price”

Data Output	Messages	Notifications
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL		
	booking_id [PK] integer	passenger_passport_number [PK] character varying (20)
		seat_number character varying (10)

Data Output	Messages	Notifications
≡+ 📄 ▼ 📋 ▼ 🗑️ 🗄️ ⬇️ 📈 SQL		
	passenger_passport_number [PK] character varying (20)	passenger_full_name character varying (100)

And 1NF table:

Data Output

Messages

Notifications

Now, there are still some tasks remaining, first we need to change departing_gate from VARCHAR to TEXT:

Query	Query History
1	ALTER TABLE booking_receipt_1nf
2	ALTER COLUMN departing_gate TYPE TEXT;

And also drop “info” column from airline tables:

Query	Query History
1	ALTER TABLE booking_receipt_1nf
2	DROP COLUMN info;